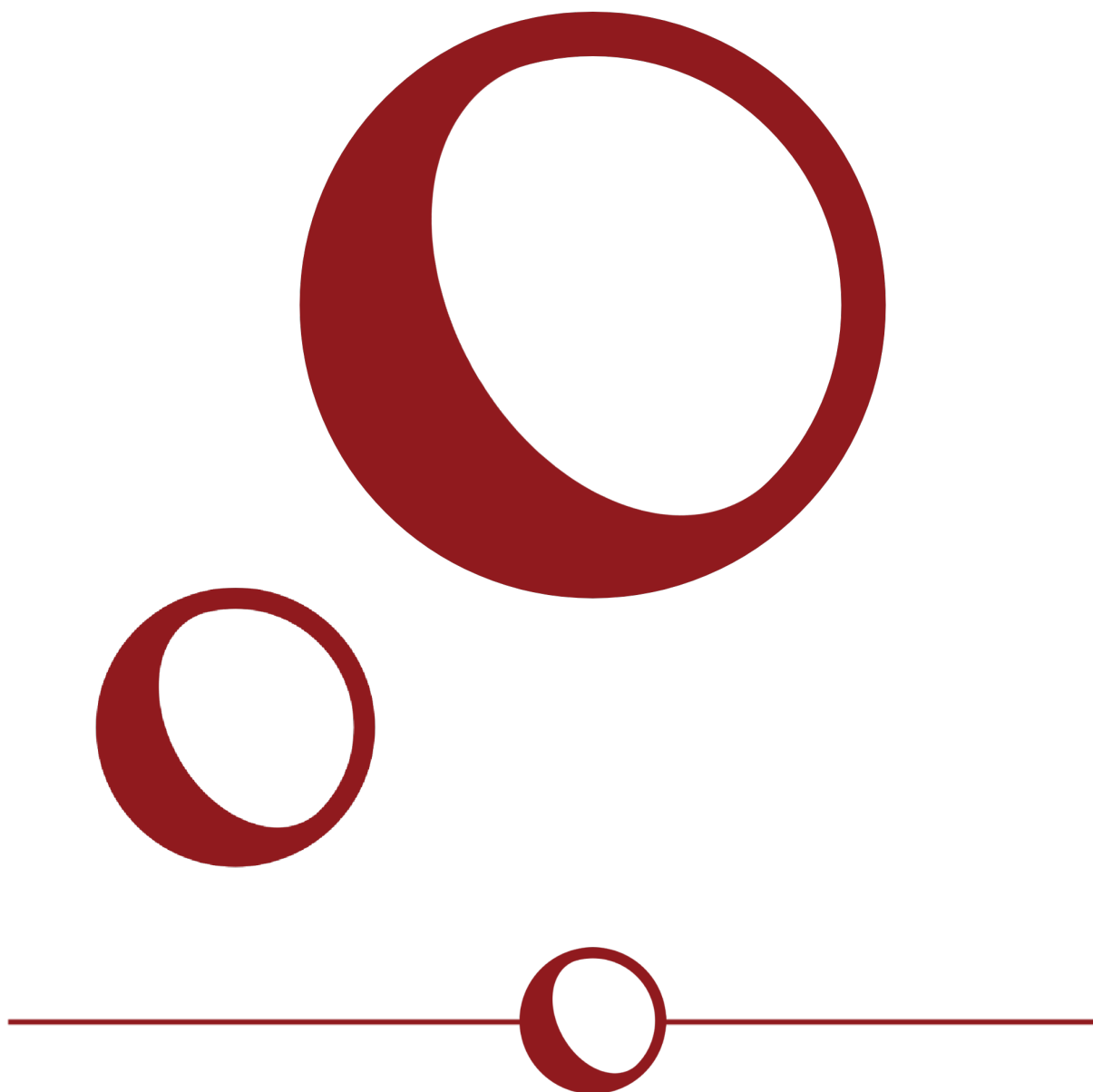




HPPC Final Project - GPU Skymion Simulation

Amalie F. Davidsen	gch817@alumni.ku.dk
Asger V. B. Andersen	lhx671@alumni.ku.dk
Bastian Holten-Møller	zxm415@alumni.ku.dk
Benjamin Siddique	vhb925@alumni.ku.dk
Julie R. Walas	jqz760@alumni.ku.dk
Rasmus Madsen	zpr147@alumni.ku.dk

March 21, 2026

1 Introduction

The Ising model is the simplest toy model of magnetism, describing magnetic spins and their interactions. It's able to reproduce many physical phenomena such as magnetic domains, phase transitions and more. However, it only describes a special anisotropic case of three dimensional spins, and therefore it misses some interesting physics. One such phenomena requiring three dimensional spins are so called skyrmions: Vortex-like spin structures in systems with DMI interactions. They are thought to be promising for use in both spintronics and Q-bits due to their stability. In this project we implement the more complete Heisenberg model, (which does have three dimensional spins), in a two-dimensional system, with the goal of simulating skyrmions. Furthermore, we implement this on a GPU and optimize the parallelization to achieve a large performance gain.

1.1 Basic Theory

The Heisenberg interaction is usually limited to the nearest $\langle j \rangle$ neighbors and a constant exchange interaction J . It is also often accompanied by the Zeeman energy of a magnetic moment in an external field, such as an applied B field in the z -direction. In this system, it is energetically beneficial for magnetic moments to align with each other and with the external field, Figure 1.

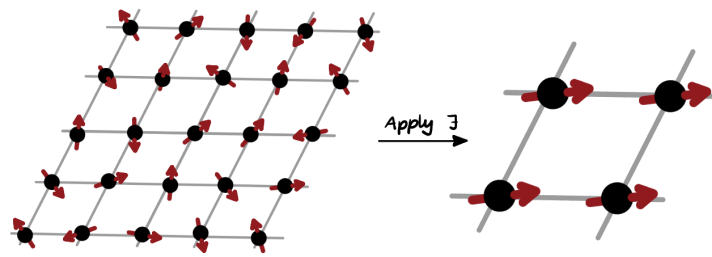


Figure 1: A lattice of randomly oriented spins (no B field), and the effect of exchange interactions.

In order to induce skyrmions, we need to add a term which incentivizes some mechanism of rotation. This is done with the Dzyaloshinskii–Moriya Interaction (DMI), which is fundamental for skyrmions:

$$\mathcal{H}_i = \underbrace{-g \mu_B B S_{iz}}_{\text{Zeeman Energy}} - \underbrace{\sum_{\langle j \rangle} J (\mathbf{S}_i \cdot \mathbf{S}_j)}_{\text{Heisenberg Hamiltonian}} - \underbrace{\sum_{\langle j \rangle} \mathbf{D}_{ij} \cdot (\mathbf{S}_i \times \mathbf{S}_j)}_{\text{DMI Interaction.}} \quad \text{Where } \mathbf{D}_{ij} \equiv D \mathbf{r}_{ij}$$

Here g is the Landé g -factor, μ_B is the Bohr magneton and B is the magnetic field in the z -direction. $\mathbf{S}_i$ and $\mathbf{S}_j$ are spin vectors with corresponding x , y , and z -components, i.e. S_{iz} . Finally J quantizes the exchange interaction between spins and $\mathbf{D}_{ij}$ is the DMI vector between sites i and j . This type of DMI leads to so called Bloch Skyrmions (as opposed to other types such as Néel skyrmions).

1.2 Basic Implementation

The standard implementation of an Ising/Heisenberg model uses a Monte Carlo simulation with the so called Metropolis criterion. In short, first you initialize a grid of randomly oriented spins. Next you randomly pick a spin-site and calculate it's current energy in relation to its neighbours. Then you induce a random spin-flip and calculate the energy again. Finally, you compare the difference in energy, ΔE , with a randomly generated thermal excitation energy. If the energy difference is less than the thermal excitation, the spin-flip succeeds, and if not it is reverted.

This thermal excitation is called the Metropolis criterion, and stems from the Boltzmann distribution derived from the assumption that the underlying system is in thermal contact with a heat reservoir.

$$\text{Spin-flip} = \begin{cases} \text{Accept} & \text{if } e^{-\beta\Delta E} \geq P \\ \text{Reject} & \text{if } e^{-\beta\Delta E} < P \end{cases} \quad \text{Where } \beta = \frac{1}{k_B T} \text{ and } P \in [0, 1]$$

Notably, if ΔE is negative the criterion automatically succeeds. This simply describes spins aligning when energetically favourable. T represents the temperature of the system, so for instance at zero temperature no spin flips will occur after an equilibrium point has been reached, while at infinite temperature all spin flips will be accepted, which corresponds to maximum entropy. The energy mainly depends on the strength of the magnetic field, B , and interaction parameters such as J or in our case D , which determine how much energy is saved by aligning or rotating spins.

2 Parallelization and GPU Implementation

The Ising/Heisenberg model is a prime candidate for parallelization, since each update only depends on the state of the nearest neighbours. Thus, we are free to update any number of spins in parallel, as long as none of them neighbour each other. If we take this fact to the extreme, we can actually update a significant fraction (for example 1/2) of all the spins completely simultaneously!

2.1 Checkerboard Strategy

The most commonly used geometry is a square lattice of sites, i.e. a grid. If you color half of these grid sites red and the other half black it forms a checkerboard pattern. Notice how the red squares only neighbor black squares, and vice versa. So in a simulation it would be possible to update all the red squares simultaneously, and then all the black squares.

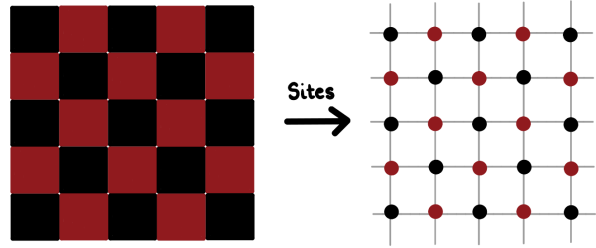


Figure 2: Illustration of a checkerboard pattern and how this reflects the spin site locations.

This configuration is illustrated on figure 2, where the sites are also shown as nodes on a grid. This strategy is perfectly suited for running on a GPU, since so many calculations can be run in parallel, and for the square grid is easy to map it onto a GPU with some simple indexing.

Of course, not all grids are created equally, and in condensed matter physics, other grid layouts than the square one are often of interest. Specifically, triangular lattices and so called Kagome lattices are particularly appealing to us. For these more complicated grids, there are two difficulties compared to the orthonormal square grid. First, we need to figure out a new colouring scheme, such that no neighbours share the same colour. Second, it is not obvious how to index these sites, which we will need to do carefully, so we can map them to the GPU.

The first challenge is known as the map coloring problem, which describes how many colours are needed to colour a graph, such that no connected nodes share a colour. A famous result of which is the "Four colours theorem", which states that no more than four colours are necessary in 2D.

For the square grid only two colours are needed, whilst all four are needed for the triangular graph. The Kagome lattice is less constrained, so we only need three colours, as illustrated in Figure 3.

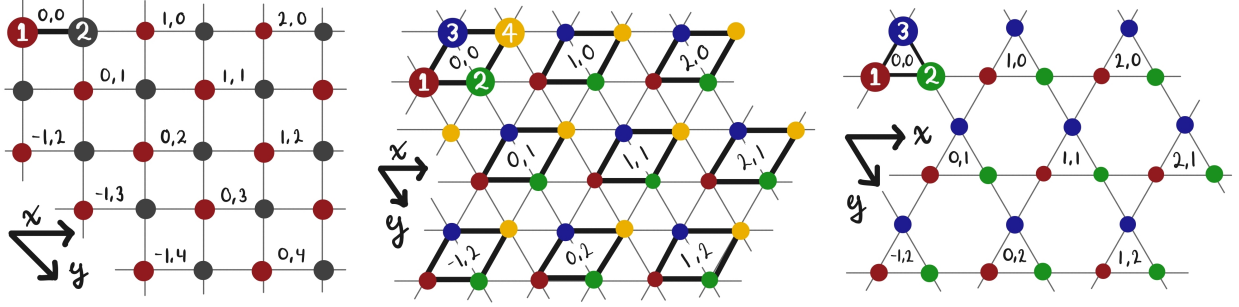


Figure 3: The unit cell layout for the square, triangular and Kagome lattices.

We can solve the second problem by defining a unit cell containing one node of each colour. For the square lattice, this is just two nodes, while for the triangular lattice it is a rhombus. Next we define an x - and a y -axis, which we can use to index the unit cells. Finally a third index can be used to iterate over the different colours in each unit cell. This way, all three grid types reduce to a two dimensional grid, which we can map to the GPU. This approach was the most generalized indexing we came up with, but some of the code is indexed slightly differently. For instance the most optimized code is hard-coded for the square lattice, and just uses normal x - and y -axes.

2.2 Code Outline

In the spirit of the course, we decided to work on two branches of the code in *parallel*. The first code focused on implementing the different lattice structures, while the other code focused on maximizing performance for just the square lattice. Although the outline of both codes are similar:

Initialize an N^2 lattice of spin sites. Each spin should start with a random orientation, but we would also like reproducible results. To do this we use a seeded random number generator, which is initialized with `init_rng_kernel`. This assigns a random number generator to each site, such that every run uses the same sequence of numbers. Then `init_spins_kernel` uses these random numbers to generate a spin at each site. This is equivalent to generating a random point on a unit sphere, which can be uniformly done with a random z -value between $[-1, 1]$, from which a polar angle θ is calculated. Afterwards a random azimuthal angle φ between $[0, 2\pi]$ is used to calculate $\sin(\varphi)$ and $\cos(\varphi)$. This is then saved as cartesian coordinates in x , y and z arrays.

Run the actual simulation. We run for some amount of **steps/sweeps**, where each step/sweep updates every site once. For each step we iterate over the different colors, so for instance the square lattice has two iterations: the black sites and the red sites. For each of these we launch `mc_sweep_kernel`. This kernel first generates a new random spin at each site. It then calculates the energy difference ΔE and compares the Boltzmann factor $e^{-\beta\Delta E}$ to a random number between $[0, 1]$. The Metropolis criterion is applied and the spin flip is either accepted or rejected.

Output If `saveframe` is set to 1, then for every 1000 steps a snapshot of all the spins is written to a file. This can then be used for animating the time evolution of the system. The final state of the system is also saved, and the total final energy is calculated and can be used as a checksum.

3 Benchmarking and Optimization

The performance-maxxing code went through 5 iterations, where the last 3 were based on version 2, see Table 1. We benchmarked each of these and compared them to the original sequential code.

Version	Name	Description
0	Serial	Initial sequential code which updates the sites in a checkerboard pattern, such that each step corresponds to an update of all sites.
1	First Imp.	The first parallel implementation of our system, where the each of the checkerboard colours updates simultaneously on the GPU.
2	Float	Doubles have been changed to floats along with simplified trigonometric functions and modulus has been changed to conditional addition.
3	No Trig.	All orientations are generated in purely cartesian coordinates.
4	Shared	Loads data into shared memory if size doesn't exceed 48kB.
5	8B Rand.	The 64 Byte random number generators is changed to an 8 Byte one.

Table 1: Performance optimized code versions with short descriptions.

For each of these versions we will look at asymptotic performance plots, and then we will use NVIDIA Nsight to analyze the performance and profile the code. One of our group members happened to have access to a rather powerful NVIDIA RTX 5070 Ti GPU, and since performance on ERDA was very inconsistent, we preferred to use this machine whenever possible.

3.1 Asymptotic Performance

To gain some initial insight into the performance of each version, and in order to determine an optimal lattice size for full GPU utilization, we plot the time per step for the `mc_sweep_kernel`. This function is run twice for each step, since both the red and the black sites have to be updated.

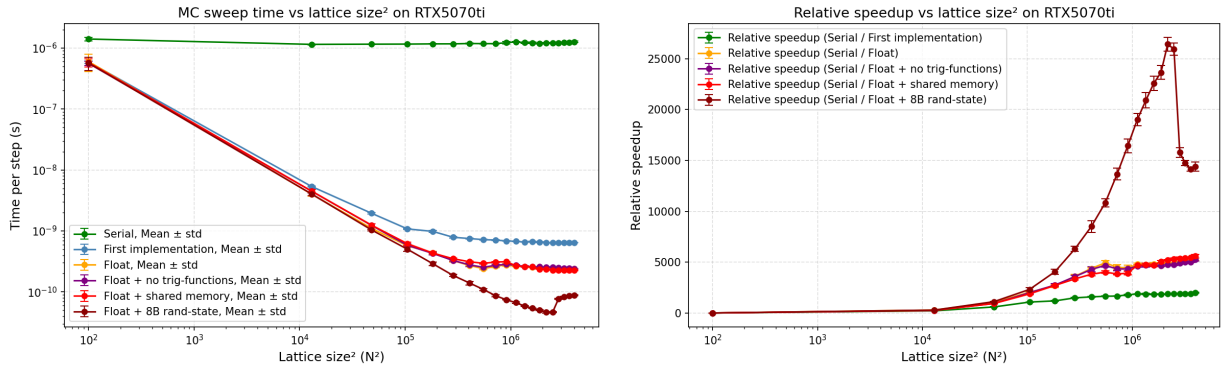


Figure 4: Performance for `mc_sweep_kernel` (left) and the speedup for the full code (right).

The immediate take-away from Figure 4 is that multiple large performance gains are made. The initial implementation on the GPU already results in about a 2000 times speedup for large grids. The second version with floats further improves this to an over 5000 times speedup. The no-trig and shared memory versions don't seem to perform any better than the float version. Finally, a huge leap in performance is gained by reducing the size of the random number generators.

It seems like the GPU becomes saturated around a lattice size of 10^6 , which corresponds to $N = 1000$. However, this is not the case for the 8B Rand. version, where performance increases up until a grid size of around $2.4 \cdot 10^6$, ($N \approx 1600$). Beyond this point performance dips quite dramatically, although it remains at around a three times speedup from the float version.

Beyond the immediate performance increase from running on a GPU, the other performance leaps seem to be connected with minimizing allocated memory usage. In the first implementation around 6 doubles per site are allocated on the GPU, as well as the 64 byte random number generator (rng). For a lattice size of 10^6 , this results in about 112 MB of allocated memory. In the float version, we only allocate 3 floats as well as the 64 Byte rng, so this is reduced to around 76 MB. Finally, we switch to the 8 Byte rng, which lowers it all the way to around 20 MB.

This final reduction in allocated memory allows the GPU to store the problem in the shared L2 cache, which is 48 MB for an RTX 5070 Ti, and leads to a huge performance gain. This also explains the sudden drop in performance at a grid size of $2.4 \cdot 10^6$, since this is the exact point where the allocated memory exceeds 48 MB, which forces the data onto the much slower global GPU memory.

We can also use these numbers to calculate the maximum runnable grid size. Our GPU has 16 GB of global memory, which we can divide with the 20 Bytes per site, to get a maximum grid size of $\approx 0.8 \cdot 10^9$, which corresponds to $N \approx 28\,000$. And sure enough, when we run this on the GPU, we can see in task manager, that the entire GPU memory is in use.

3.2 Profiling

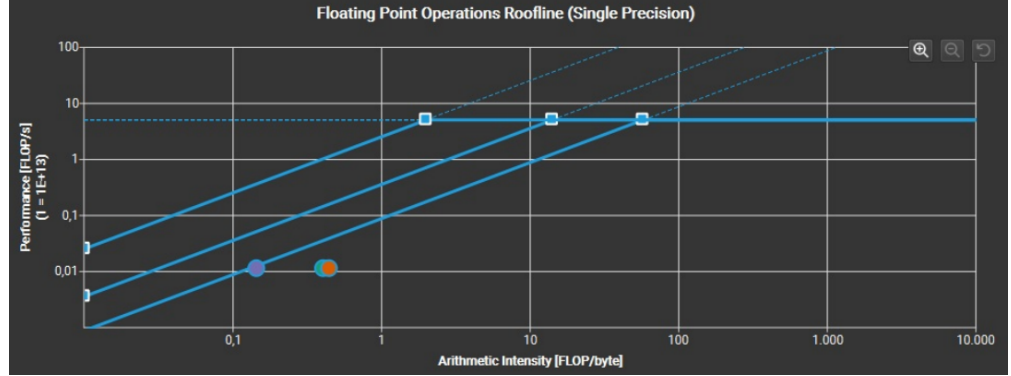
We used the NVIDIA Nsight Compute profiler to analyze the performance of our kernels across different implementations. The central kernel in each implementation is the `mc_sweep_kernel`, so this will be the main subject of study. We analyzed a typical run of this kernel across implementations on grids of size 1000^2 , and have listed some key parameters in Table 2.

Version	Loaded from DRAM [MB]	L1 hit rate [%]	L2 hit rate [%]
1	45.56	65.46	72.08
2	44.03	62.48	73.51
3	44.05	63.55	72.48
4	44.05	38.81	80.54
5	20.01	43.64	67.44

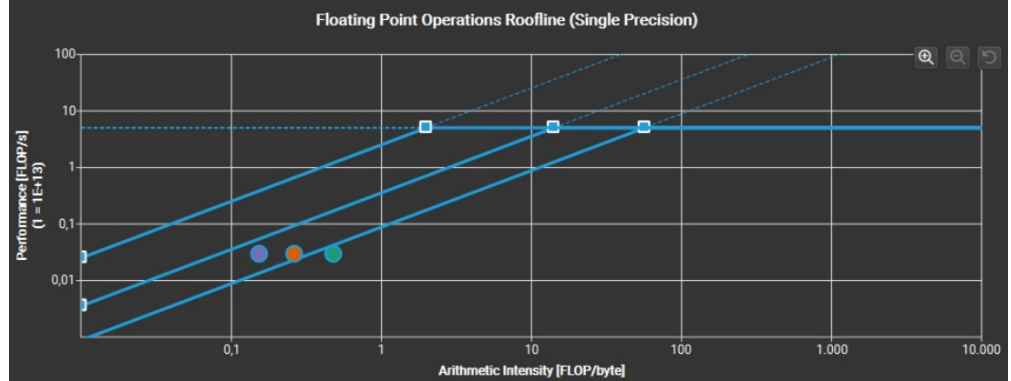
Table 2: Memory parameters from profiling different versions of code.

We generally have very high cache hit rates. Note that the hit rates compound, such that for the first version, $65.46\% + (1 - 65.46\%) \cdot 72.08\% \approx 90\%$ of memory fetches are from the cache. The hit rates drop off significantly for our shared memory layout (version 4) and the reduced 8-bit implementation (version 5). For the latter it is likely because the memory usage is just much smaller, and therefore the cache fetches are also significantly smaller. For the former it is a result of the fetches that would usually go towards the cache, are instead obtained from the faster shared block memory. These decreases in cache hit rates therefore do not correspond to worse performance.

Version 1



Version 4



Version 5

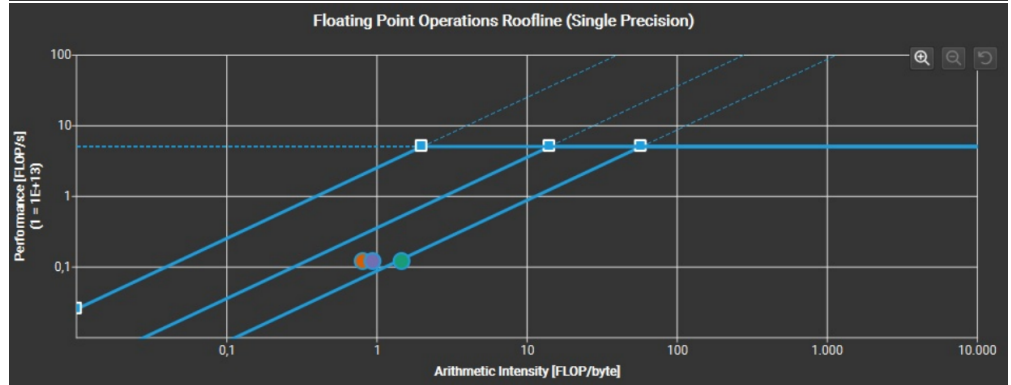


Figure 5: Roofline plots for versions 1, 4 and 5 for grids of size 1000^2 . The three rooflines represent, from left to right, the theoretical bandwidth and performance limits of L1, L2 and DRAM. Our results are given by the dots below the rooflines (L1 = purple, L2 = orange, DRAM = green).

We seem to be significantly memory bounded, meaning that we are capped by how fast the GPU is able to fetch data. This is closely tied to our implementation, since we need many memory fetches when flipping half the grid sites at once. In version 4 we tried to improve bandwidth, by utilizing shared memory within the blocks (so that multiple threads can access the same neighbor properties in the grid). But as seen in Figure 5, this had negligible effects (also on even larger sized grids).

We do come very close to optimal performance, given our bandwidth cap, in version 5, where the DRAM dot lies exactly on the diagonal of its roofline. This is a significant improvement from version 1, where the DRAM is stuck waiting for memory fetches, sitting idle while it could be doing more operations. To improve one should consider investigating shared memory more thoroughly, since it should be possible to increase the arithmetic intensity, by decreasing the number of fetched bytes.

4 Physical Results

The number of steps/sweeps to run the simulation for should lead to a converged energy, since we want to optimize for a ground state. We also use the energy convergence as an approximate checksum. In figure 6 we see the energy convergence for several versions of code. They are not in perfect agreement, which we ascribe to the fact that the data types between codes are not the same. For example, the sequential version uses doubles, while the parallel versions use floats. The energy converges within a short interval for all codes, and the spin configurations are visually similar. We conclude that a step number of $\sim 200\,000$ is enough for convergence.

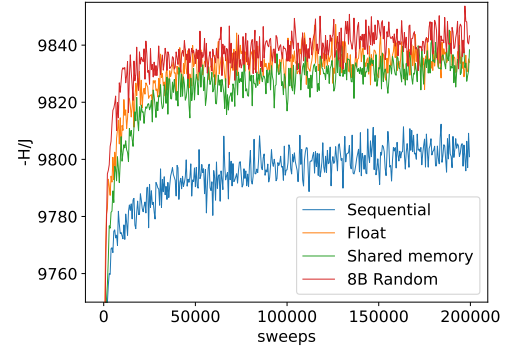


Figure 6: Total Hamiltonian energy as a function of steps for different versions.

4.1 Phases

Depending on the strength of the parameters D and B , different phases will be induced. Here we will describe three such phases, as seen in figure 9. In the top row we plot the x and y component of the spin vector in the plane and indicate the z component as the color. On the bottom row we plot $S(\mathbf{q})$ which is the Fourier transform of the the real space spin configuration defined as:

$$S(\mathbf{q}) = \frac{1}{N} \sum_{\alpha \in \{x,y,z\}} \left| \sum_i^N S_i^\alpha e^{i\mathbf{q} \cdot \mathbf{r}} \right|^2$$

$S(\mathbf{q})$ is useful for analyzing how the spins align and order, and is also the parameter measured in elastic neutron scattering experiments. If $S(\mathbf{q})$ is rotationally invariant the spin configuration does not order in a lattice like structure. The two skyrmion phases seen in Figure 9 are an example of this. If $S(\mathbf{q})$ is not rotationally invariant, the spins are ordered in a lattice like structure.

Skyrmion phase I (Figure 9, Left)

When ferromagnetic interactions are allowed to dominate with small contributions from the DMI interaction and B-field, fewer and larger skyrmions form. Due to the width of these skyrmions and the finite size of the grid, $|\mathbf{q}|$ for the interactions overlaps with the ferromagnetic Bragg peak in the center. To further resolve these larger skyrmions a larger grid could be used.

Skyrmion phase II (Figure 9, Middle)

This phase is induced with a strong DMI interaction and a strong magnetic field, and is characterized by small tight Skyrmions. The ferromagnetic order is still conserved, since the Bragg peak is still visible in the center. The ring characteristic of the Skyrmion phase is also visible, but since the vortexes are small, $|\mathbf{q}|$ is generally larger than in the large Skyrmion phase. The features are diffuse, and have rotational symmetry, which shows, (as is also visible in real space), that the skyrmions do not form a lattice themselves and vary in size. It could be interesting to run the simulation for many more steps to see whether the Skyrmions eventually form a lattice.

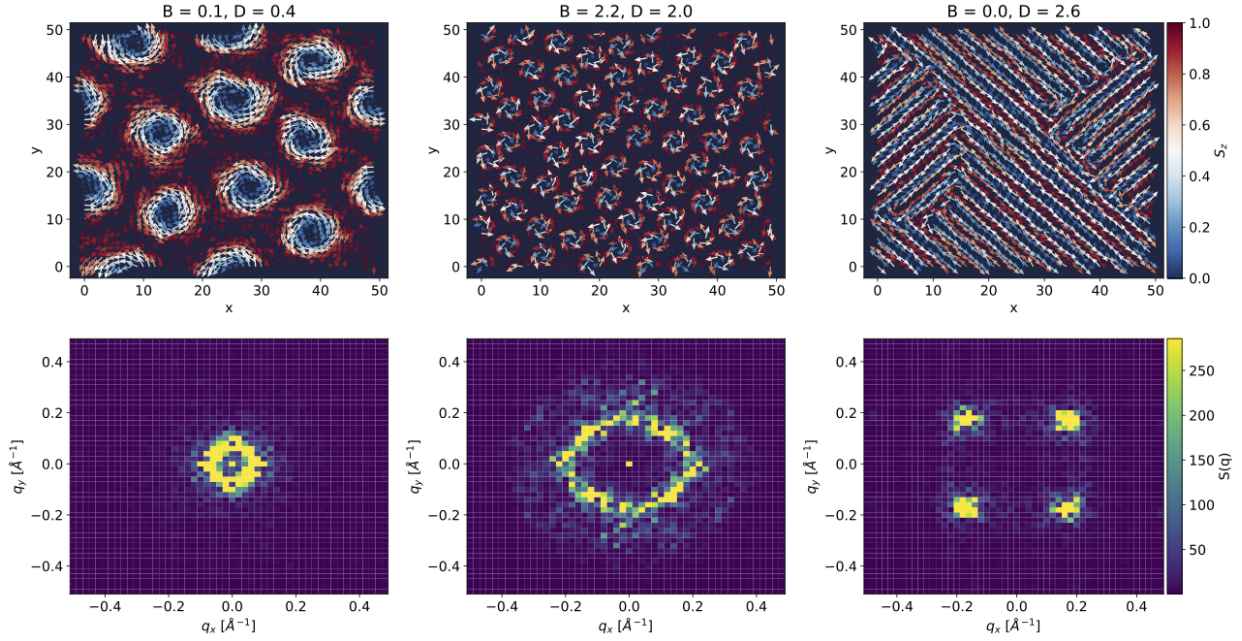


Figure 7: Top row: Real space representation of spin structures for different phases. Bottom row: Reciprocal space structure corresponding to the respective image above. Left: Large Skyrmion phase - Center: Small Skyrmion phase - Right: Chiral phase.

Chiral phase (Figure 9, Right)

In the absence of a magnetic field, a skyrmion phase does not form, regardless of the magnitude of the DMI interaction. The phase is characterized by bands of equal sign spins forming. Notably the width of the bands are comparable to the radius of the skyrmions, which is also visible in reciprocal space, since $|\mathbf{q}|$ matches the ones in the small skyrmion phase. The fourfold symmetry in real space is reflected in reciprocal space where the diffusive features are now fourfold symmetric.

4.2 Phase diagrams

Determining whether a spin configuration is a skyrmion phase is quite difficult to quantify, and could be a whole project in itself. From Figure 9 we see an important pattern. The skyrmion phases are all spherically symmetrical in reciprocal space, while the chiral phase has a fourfold symmetry, and the ferromagnetic phase is of course peaked at $q = 0$. Therefore we transform to polar coordinates, mask the signal at $q = 0$ and integrate along $|q|$ leaving us with an angular distribution, $S(\varphi)$. From this we calculate the standard deviation of $S(\varphi)$, which we call σ .

In Figure 8, σ is normalized to the maximal value in order to compare lattices. This qualitatively gives a good indication of the different phases in the diagrams. The ferromagnetic phase will have almost zero standard deviation, since we mask the peak at $q = 0$, and the chiral phase will give a high standard deviation. This is obviously quite a crude method, and in a more extensive study one could fit a model to the nonintegrated $S(\mathbf{q}_x, \mathbf{q}_y)$ using multiple Gaussians and an analytical ferromagnetic form factor. Such a fit could for instance provide insight into the symmetry of the lattice and presence of ferromagnetic interactions. Another more advanced method could be employing machine learning methods to classify the phases, which we saw in one of our references.

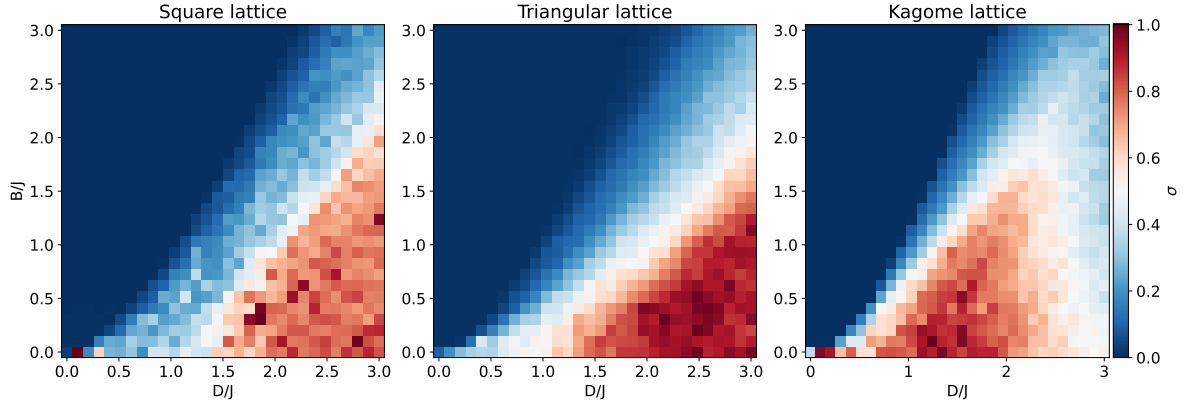


Figure 8: Phase Diagrams. Left: Square lattice. Center: Triangular lattice. Right: Kagome lattice.

For each lattice we ran the simulation at $T = 0.1$ K, and for 200 000 steps and a parameter sweep. All three lattices have a ferromagnetic phase (dark blue), a skyrmion phase marked by the (light blue) and a chiral phase (red). This we conclude from a visual inspection of the spin configuration. The region in which skyrmions can be stabilized seems to be narrowed in the triangular lattice, perhaps because the increased amount of nearest neighbors effectively increases the DMI interactions. This makes the skyrmions small at relatively low values of D .

In the Kagome lattice, the region in which skyrmions can be stabilized is even smaller, because the triangles and hexagons compete in forming the skyrmion lattice. In the high D region, the chiral phase turns into something which looks like a disordered phase, but which still has weak correlations in $S(\mathbf{q})$. This could be due to the two competing length scales of the hexagons and the triangles. We could also be witnessing a form of frustration, where each triangle wants to form a vortex, but since they are connected to another triangle, an opposite vortex might compete against it.

Our estimation is also rather crude. Since we are integrating over areas of $S(\mathbf{q})$, we are not utilizing the full structure of $S(\mathbf{q})$ to identify different phases. In further studies we could focus on lower values of D for the triangular and kagome lattice. It would also be interesting to investigate how the typical length scale of the skyrmion phases changes as a function of D and B . The typical length scale could be extracted from $S(\mathbf{q})$ as the values of q where the intensity is the largest.

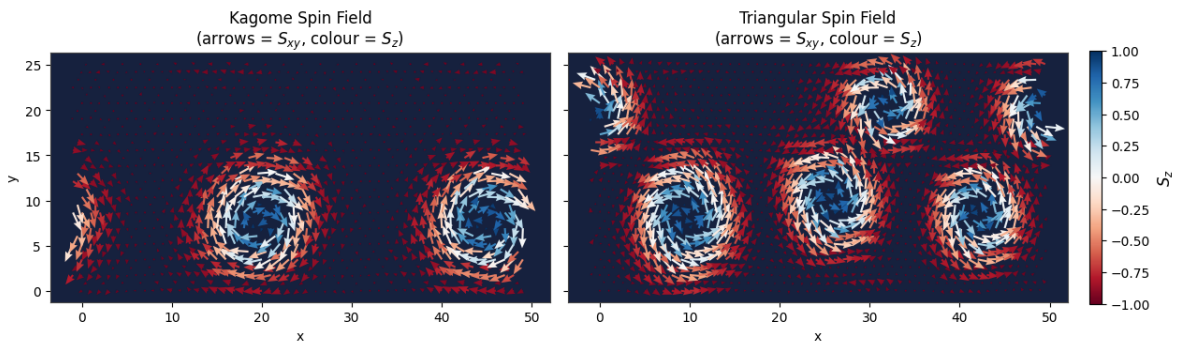


Figure 9: Some more Skyrmions because they are pretty. Left: A Skyrmion phase in the Kagome lattice. Right: A Skyrmion phase in the triangular lattice. Simulations had $D = 0.3$ and $B = 0.08$.

5 Conclusion

In this project, we successfully implemented a GPU-accelerated Monte Carlo simulation of the Heisenberg model with Dzyaloshinskii-Moriya Interactions, with the goal of simulating and characterizing skyrmion phases in various two-dimensional lattice structures.

On the computational side, we demonstrated that the checkerboard strategy utilizes GPU hardware very well, yielding an initial speedup for large grids of over 2000 times the sequential baseline. By switching from doubles to floats and reducing the random number generator from 64 bytes to 8 bytes, we drastically reduced the required memory allocation. This allowed the program to make very good use of the L2 cache, and yielded a maximum speedup exceeding 25000. Roofline analysis confirmed that our implementation is mainly memory bounded, so further performance gains might be achievable through a better implementation of shared memory.

On the physics side, we successfully reproduced and characterized three distinct phases: ferromagnetic, skyrmion, and chiral. Each of these appear in all three implemented lattice types: square, triangular, and Kagome. Using the standard deviation of the reciprocal-space spin structure factor as a phase indicator, we constructed phase diagrams for the strength of the DMI parameter D vs the applied B field. All three lattice types have similar phase diagrams, but they are distinct. Future work could explore more rigorous phase classification, for instance by using machine learning methods, which was seen in one of our reference papers.

There is one large limitation of this project, and that is the report format. The beauty of our physical results really shine when they are allowed to be animated. So we are looking forward to showing some very pretty animations at our presentation on Wednesday!

6 Appendix

6.1 serial.cpp

```

#define _USE_MATH_DEFINES
#include <vector>
#include <iostream>
#include <chrono>
#include <cmath>
#include <numeric>
#include <random>
#include <fstream>
#include "random_number.h"

double J = 1; //units of mev
double D = 1; //units of mev
double B = 0;
double T = 5; //units of kelvin
double beta = 1.0/(0.086*T);
int size = 10;
int seed = 0;
bool gif = false;
int steps = 1000;

class Spin{
public:
    double theta;
    double phi;
    std::vector<int> neighbours {4};
    double local_energy;

    // Constructor
    Spin(double theta, double phi)
        : theta(theta), phi(phi), neighbours(4), local_energy(0.0) {}

    // Default constructor (zero-initialized)
    Spin() : theta(0.0), phi(0.0), neighbours(4), local_energy(0.0) {}
};

class Lattice{
public:
    int size;
    std::vector<Spin> spins;
    double energy = 0;
    std::vector<double> mag = {0.0, 0.0, 0.0};

    Lattice(int s) : size(s), spins(s * s) {}

    Spin& at(int row, int col){
        return spins[row*size + col];
    }

    double calc_tot_energy(){
        energy = 0;
        for (size_t i = 0; i < spins.size(); i++){
            energy += 0.5 * spins[i].local_energy;
        }
        return energy;
    }

    std::vector<double> calc_tot_mag(){
        mag = {0.0, 0.0, 0.0};
        for (size_t i = 0; i < spins.size(); i++){
            mag[0] += sin(spins[i].theta)*cos(spins[i].phi);
            mag[1] += sin(spins[i].theta)*sin(spins[i].phi);
            mag[2] += cos(spins[i].theta);
        }
        return mag;
    }

    void save_spins(const std::string& filename) {
        std::ofstream file(filename);

        if (!file.is_open()) {
            std::cerr << "Error: could not open file " << filename << "\n";
            return;
        }
    }
}

```

```

// Header
file << "index,row,col,theta,phi\n";

for (size_t i = 0; i < spins.size(); i++) {
    int row = i / size;
    int col = i % size;
    file << i << ", "
        << row << ", "
        << col << ", "
        << spins[i].theta << ", "
        << spins[i].phi << "\n";
}

file.close();
}

};

//double dmi_int(double thetai, double thetaj, double phii, double phij){
//    return D*(sin(thetai)*cos(thetaj)*(sin(phii) - cos(phii)) + cos(thetai)*sin(thetaj)*( cos(phij)- sin(phij)));
//};

//double diag_int(double thetai, double thetaj, double phii, double phij){
//    return J*(sin(thetai) *sin(thetaj)*cos(phii - phij) + cos(thetai) *cos(thetaj));
//};

//double dmi_int(Spin &spini, Spin &spinj){
//    return D*(sin(spini.theta)*cos(spinj.theta)*(sin(spini.phi) - cos(spini.phi)) + cos(spini.theta)*sin(spinj.theta)*
//};

double cross_x(Spin &spini, Spin &spinj){
    // Si X Sj dot xhat
    return D*(sin(spini.theta)*sin(spini.phi)*cos(spinj.theta) - cos(spini.theta)*sin(spinj.theta)* sin(spinj.phi));
}
double cross_y(Spin &spini, Spin &spinj){
    // Si X Sj dot yhat
    return D*( cos(spini.theta)*sin(spinj.theta)* cos(spinj.phi) - sin(spini.theta)*cos(spini.phi)*cos(spinj.theta));
}

double diag_int(Spin &spini, Spin &spinj){
    return J*(sin(spini.theta) *sin(spinj.theta)*cos(spini.phi - spinj.phi) + cos(spini.theta) *cos(spinj.theta));
};

double zeeman(Spin &spini){
    return B*cos(spini.theta);
}

double all_int(Spin &spini, Lattice &lattice){
    double dE = 0 ;
    dE -= diag_int(spini, lattice.spins[spini.neighbours[0]]) - cross_y(spini, lattice.spins[spini.neighbours[0]]); //
    dE -= diag_int(spini, lattice.spins[spini.neighbours[1]]) + cross_y(spini, lattice.spins[spini.neighbours[1]]);
    dE -= diag_int(spini, lattice.spins[spini.neighbours[2]]) - cross_x(spini, lattice.spins[spini.neighbours[2]]);
    dE -= diag_int(spini, lattice.spins[spini.neighbours[3]]) + cross_x(spini, lattice.spins[spini.neighbours[3]]);

    return dE;
};

void config_sim(int argc, char **argv){
    for (int i = 1; i < argc; i += 2){
        std::string arg = argv[i];
        //std::cout << argv[i] << std::endl;
        if (arg == "-J"){J = std::stod(argv[i+1]);}
        else if (arg == "-D"){D = std::stod(argv[i+1]);}
        else if (arg == "-T"){T = std::stod(argv[i+1]); beta = 1.0/(0.086*T);}
        else if (arg == "-B"){B = std::stod(argv[i+1]);}
        else if (arg == "-size"){size = std::stod(argv[i+1]);}
        else if (arg == "-steps"){steps = std::stod(argv[i+1]);}
        else if (arg == "-seed"){seed = std::stoi(argv[i+1]);}
        else if (arg == "-gif"){gif = (std::stoi(argv[i+1]) == 1) ? true: false;}
        else {std::cout << "Argument not recognized" << argv[i] << std::endl;};
    }
};

```

```

void setup_sim(Lattice &lattice){
    int N = size*size;
    std::vector<float> random_thetas = randomArray(seed, N, -1.0f, 1.0f);
    std::vector<float> random_phis = randomArray(seed, N, 0.0f, 1.0f);
    for (size_t i = 0; i < lattice.spins.size(); i++){
        lattice.spins[i].theta = acos(random_thetas[i]);
        lattice.spins[i].phi = random_phis[i] * 2.0 * M_PI;
        int row = i/size;
        int column = i%size;
        lattice.spins[i].neighbours[0] = ((row + 1)%size)*size + column;
        lattice.spins[i].neighbours[1] = ((row - 1 + size)%size)*size + column;
        lattice.spins[i].neighbours[2] = (row)*size + (column + 1)%size;
        lattice.spins[i].neighbours[3] = (row)*size + (column - 1 + size)%size;
    }

    for (size_t i = 0; i < lattice.spins.size(); i++){
        lattice.spins[i].local_energy = all_int(lattice.spins[i], lattice) + zeeman(lattice.spins[i]);
    }
    lattice.calc_tot_energy();
};

void update_spin(Lattice &lattice, int j, float &random_theta, float &random_phi, float &random_e){
    Spin new_spin_i(acos(random_theta), random_phi * 2.0 * M_PI);

    Spin old_spin_i = lattice.spins[j];
    new_spin_i.neighbours = old_spin_i.neighbours;
    new_spin_i.local_energy = all_int(new_spin_i, lattice) + zeeman(new_spin_i);
    double dH = new_spin_i.local_energy - old_spin_i.local_energy;

    if (dH < 0 || std::exp(-dH*beta) > random_e){
        lattice.spins[j] = new_spin_i;
        lattice.energy += dH;

        // Update neighbours' local energy
        for (int n : new_spin_i.neighbours){
            lattice.spins[n].local_energy = all_int(lattice.spins[n], lattice) + zeeman(lattice.spins[n]);
        }
    }
};

void iterate(Lattice &lattice){
    // One step consists of iterating through all the atoms.
    for(int i = 0; i < steps ; i++){
        // iteration in chessboard pattern
        std::vector<float> random_thetas = randomArray(seed + i, size*size, -1.0f, 1.0f);
        std::vector<float> random_phis = randomArray(seed + i + steps, size*size, 0.0f, 1.0f);
        std::vector<float> random_e = randomArray(seed + i + 2*steps, size*size, 0.0f, 1.0f);

        // update all parity-0 sites: (row+col) % 2 == 0
        for (int idx = 0; idx < size*size; idx++){
            int row = idx / size;
            int col = idx % size;
            if ((row + col) % 2 == 0)
                update_spin(lattice, idx, random_thetas[idx], random_phis[idx], random_e[idx]);
        }
        // update all parity-1 sites: (row+col) % 2 == 1
        for (int idx = 0; idx < size*size; idx++){
            int row = idx / size;
            int col = idx % size;
            if ((row + col) % 2 == 1)
                update_spin(lattice, idx, random_thetas[idx], random_phis[idx], random_e[idx]);
        }
        if (gif && i%100 == 0){
            std::ofstream gifFile("frames.csv", std::ios_base::app);
            for (size_t i = 0; i < lattice.spins.size(); i++){
                gifFile << lattice.spins[i].theta << "," << lattice.spins[i].phi << "\n";
            }
        }
    }
};

int main(int argc, char **argv) {
    config_sim(argc, argv);
    if (gif){
        std::ofstream gifFile("frames.csv");
    }
}

```

```
// header consist of theta, phi and value of the gridSize
gifFile << "theta,phi," << size << "\n";
std::cout << "Creating_GIF_frames..." << std::endl;
}
auto begin = std::chrono::steady_clock::now();
Lattice lattice(size);
setup_sim(lattice);
iterate(lattice);
std::cout<< lattice.calc_tot_energy() <<std::endl;
auto end = std::chrono::steady_clock::now();
std::cout << (end - begin).count()/1000000000.0/(size*size*steps) << "_sec/step_serial" << std::endl;
std::cout << "Total_time:_ " << (end - begin).count()/1000000000.0 << "_sec" << std::endl;
// save final frame
if (gif){
    std::ofstream gifFile("frames.csv", std::ios_base::app);
    for (size_t i = 0; i < lattice.spins.size(); i++){
        gifFile << lattice.spins[i].theta << "," << lattice.spins[i].phi << "\n";
    }
    gifFile.close();
}
lattice.save_spins("test.csv");
return 0;
}
```

6.2 8B Rand.cu

```

#define _USE_MATH_DEFINES
#include <cmath>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

#include <cuda_runtime.h>

// ——— Simulation parameters ———
static double J = 1.0;
static double D = 1.0;
static double B = 0.0;
static double T = 5.0;
static double beta = 1.0 / (0.086 * T);
static int size = 10;
static int steps = 1000;
static int seed = 0;
static bool saveframe = false;
static std::string outfile = "timing_results_float_8B_2.csv";

static void check_cuda(cudaError_t code, const char* context) {
    if (code != cudaSuccess) {
        fprintf(stderr, "%s: %s\n", context, cudaGetErrorString(code));
        exit(EXIT_FAILURE);
    }
}

#define M_PI_F 3.14159265f

// ——— PCG32 — 8-byte state, high-quality 32-bit output ———
// State: one uint64 per site (8 bytes vs 64 bytes for Philox).
struct Pcg32 { unsigned long long state; };

__host__ __device__ inline unsigned int pcg32_next(Pcg32& rng) {
    unsigned long long old = rng.state;
    rng.state = old * 6364136223846793005ULL + 1442695040888963407ULL;
    unsigned int xorshifted = (unsigned int)(((old >> 18u) ^ old) >> 27u);
    unsigned int rot = (unsigned int)(old >> 59u);
    return (xorshifted >> rot) | (xorshifted << ((-rot) & 31u));
}

__host__ __device__ inline Pcg32 pcg32_seed(unsigned long long seed, unsigned long long idx) {
    unsigned long long s = seed + idx * 6364136223846793005ULL;
    s = (s ^ (s >> 30)) * 0xbf58476d1ce4e5b9ULL;
    s = (s ^ (s >> 27)) * 0x94d049bb133111ebULL;
    s = s ^ (s >> 31);
    Pcg32 rng; rng.state = s;
    pcg32_next(rng); pcg32_next(rng);
    return rng;
}

__host__ __device__ inline float pcg32_float(Pcg32& rng) {
    return (pcg32_next(rng) >> 8) * (1.0f / 16777216.0f);
}

// Fast periodic neighbour index: avoids modulo by using conditional add.
// Valid when row/col are already in [0, n-1].
__device__ inline int nbr(int base, int delta, int n) {
    int v = base + delta;
    if (v < 0) v += n;
    if (v >= n) v -= n;
    return v;
}

// ——— Initialize PCG32 states ———
__global__ void init_rng_kernel(Pcg32* rng_states, unsigned long long rng_seed, int n_sites) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n_sites) return;

```

```

    rng_states[idx] = pcg32_seed(rng_seed, (unsigned long long)idx);
}

// ----- Initialize spins - float intrinsics -----
__global__ void init_spins_kernel(
    float* x, float* y, float* z,
    Pcg32* rng_states, int n_sites) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n_sites) return;

    Pcg32 state = rng_states[idx];
    float r0 = pcg32_float(state);
    float r1 = pcg32_float(state);
    rng_states[idx] = state;

    float pz = 2.0f * r0 - 1.0f;
    float sin_theta = sqrtf(fmaxf(0.0f, 1.0f - pz * pz));
    float sph1, cphi;
    __sincosf(r1 * 2.0f * M_PI_F, &sph1, &cphi);

    x[idx] = cphi * sin_theta;
    y[idx] = sph1 * sin_theta;
    z[idx] = pz;
}

// ----- MC kernel - compute-optimised -----
__global__ void mc_sweep_kernel(
    float* x, float* y, float* z,
    Pcg32* rng_states,
    int n,
    int parity,
    float j, float d, float b,
    float inv_temperature) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int n_half = n * (n / 2);
    if (tid >= n_half) return;

    int row = tid / (n / 2);
    int half_col = tid % (n / 2);
    int col = 2 * half_col + ((row & 1) ^ parity);
    int site = row * n + col;

    Pcg32 state = rng_states[site];
    float r0 = pcg32_float(state);
    float r1 = pcg32_float(state);
    float r2 = pcg32_float(state);
    rng_states[site] = state;

    float pz = 2.0f * r0 - 1.0f;
    float sin_theta = sqrtf(fmaxf(0.0f, 1.0f - pz * pz));
    float sph1, cphi;
    __sincosf(r1 * 2.0f * M_PI_F, &sph1, &cphi);
    float px = cphi * sin_theta;
    float py = sph1 * sin_theta;
    float accept_r = r2;

    float xi = x[site], yi = y[site], zi = z[site];

    int row_p = nbr(row, +1, n), row_m = nbr(row, -1, n);
    int col_p = nbr(col, +1, n), col_m = nbr(col, -1, n);

    float xn0 = x[row_p*n+col], yn0 = y[row_p*n+col], zn0 = z[row_p*n+col];
    float xn1 = x[row_m*n+col], yn1 = y[row_m*n+col], zn1 = z[row_m*n+col];
    float xn2 = x[row*n+col_p], yn2 = y[row*n+col_p], zn2 = z[row*n+col_p];
    float xn3 = x[row*n+col_m], yn3 = y[row*n+col_m], zn3 = z[row*n+col_m];

    float dx = px - xi, dy = py - yi, dz = pz - zi;
    float delta_h =
        - j*(dx*xn0 + dy*yn0 + dz*zn0) + d*(dz*xn0 - dx*zn0)
        - j*(dx*xn1 + dy*yn1 + dz*zn1) - d*(dz*xn1 - dx*zn1)
        - j*(dx*xn2 + dy*yn2 + dz*zn2) + d*(dy*zn2 - dz*yn2)
        - j*(dx*xn3 + dy*yn3 + dz*zn3) - d*(dy*zn3 - dz*yn3)
        + b * dz;

    bool take = (delta_h < 0.0f) || (__expf(-delta_h * inv_temperature) > accept_r);

    if (take) {
        x[site] = px;
        y[site] = py;
        z[site] = pz;
    }
}

```

```

    }
}

static void config_sim(int argc, char** argv) {
    for (int i = 1; i + 1 < argc; i += 2) {
        std::string arg = argv[i];
        if (arg == "--J") {
            J = std::stod(argv[i + 1]);
        } else if (arg == "--D") {
            D = std::stod(argv[i + 1]);
        } else if (arg == "--T") {
            T = std::stod(argv[i + 1]);
            beta = 1.0 / (0.086 * T);
        } else if (arg == "--B") {
            B = std::stod(argv[i + 1]);
        } else if (arg == "--size") {
            size = std::stoi(argv[i + 1]);
        } else if (arg == "--steps") {
            steps = std::stoi(argv[i + 1]);
        } else if (arg == "--seed") {
            seed = std::stoi(argv[i + 1]);
        } else if (arg == "--saveframe") {
            saveframe = (std::stoi(argv[i + 1]) == 1) ? true : false;
        } else if (arg == "--outfile") {
            outfile = argv[i + 1];
        } else {
            std::cout << "Argument_not_recognized_" << arg << std::endl;
        }
    }
}

static void save_spins(const std::vector<float>& x, const std::vector<float>& y, const std::vector<float>& z, int n, const std::ofstream& file(filename)) {
    if (!file.is_open()) {
        std::cerr << "Error:_could_not_open_file_" << filename << std::endl;
        return;
    }
    file << "index,row,col,sx,sy,sz\n";
    for (int i = 0; i < n * n; ++i) {
        int row = i / n;
        int col = i % n;
        file << i << "," << row << "," << col << ","
            << x[i] << "," << y[i] << "," << z[i] << "\n";
    }
}

static void save_frame(float* d_x, float* d_y, float* d_z, int n_sites, int frame) {
    std::vector<float> h_x(n_sites), h_y(n_sites), h_z(n_sites);

    check_cuda(cudaMemcpy(h_x.data(), d_x, n_sites * sizeof(float), cudaMemcpyDeviceToHost), "cudaMemcpy_D2H_x");
    check_cuda(cudaMemcpy(h_y.data(), d_y, n_sites * sizeof(float), cudaMemcpyDeviceToHost), "cudaMemcpy_D2H_y");
    check_cuda(cudaMemcpy(h_z.data(), d_z, n_sites * sizeof(float), cudaMemcpyDeviceToHost), "cudaMemcpy_D2H_z");

    double total_energy = 0.0;
    for (int i = 0; i < n_sites; ++i) {
        int row = i / size;
        int col = i % size;

        int n0 = ((row + 1 + size) % size) * size + col;
        int n1 = ((row - 1 + size) % size) * size + col;
        int n2 = row * size + ((col + 1 + size) % size);
        int n3 = row * size + ((col - 1 + size) % size);

        double xi = h_x[i], yi = h_y[i], zi = h_z[i];
        double e = 0.0;
        e -= J * (xi*h_x[n0] + yi*h_y[n0] + zi*h_z[n0]) - D * (zi*h_x[n0] - xi*h_z[n0]);
        e -= J * (xi*h_x[n1] + yi*h_y[n1] + zi*h_z[n1]) + D * (zi*h_x[n1] - xi*h_z[n1]);
        e -= J * (xi*h_x[n2] + yi*h_y[n2] + zi*h_z[n2]) - D * (yi*h_z[n2] - zi*h_y[n2]);
        e -= J * (xi*h_x[n3] + yi*h_y[n3] + zi*h_z[n3]) + D * (yi*h_z[n3] - zi*h_y[n3]);
        total_energy += 0.5 * e + B * zi;
    }

    if (saveframe) {
        std::string fileName = "frames/test_cuda" + std::to_string(frame) + ".csv";
        save_spins(h_x, h_y, h_z, size, fileName);
    }
}

// --- Append one timing row to the output CSV ---
// Format: size, time_per_step_sec

```

```

// Creates the file with a header if it doesn't exist yet.
static void append_timing(const std::string& filename, int lattice_size, double sec_per_step) {
    // Check whether the file already exists (to decide whether to write a header)
    bool write_header = false;
    {
        std::ifstream test(filename);
        write_header = !test.good();
    }

    std::ofstream file(filename, std::ios::app);
    if (!file.is_open()) {
        std::cerr << "Error: could not open timing file " << filename << std::endl;
        return;
    }

    if (write_header) {
        file << "size,sec_per_step\n";
    }
    file << lattice_size << "," << sec_per_step << "\n";
}

int main(int argc, char** argv) {
    auto begin = std::chrono::steady_clock::now();
    config_sim(argc, argv);

    int n_sites = size * size;
    int n_half = size * (size / 2);
    int threads = 256;
    int blocks_all = (n_sites + threads - 1) / threads;
    int blocks_half = (n_half + threads - 1) / threads;
    int frame = 0;

    float* d_x = nullptr;
    float* d_y = nullptr;
    float* d_z = nullptr;
    Pcg32* d_rng_states = nullptr;

    check_cuda(cudaMalloc(&d_x, n_sites * sizeof(float)), "cudaMalloc_d_x");
    check_cuda(cudaMalloc(&d_y, n_sites * sizeof(float)), "cudaMalloc_d_y");
    check_cuda(cudaMalloc(&d_z, n_sites * sizeof(float)), "cudaMalloc_d_z");
    check_cuda(cudaMalloc(&d_rng_states, n_sites * sizeof(Pcg32)), "cudaMalloc_d_rng_states");

    init_rng_kernel<<<blocks_all, threads>>>(d_rng_states, (unsigned long long)seed, n_sites);
    check_cuda(cudaGetLastError(), "init_rng_kernel_launch");
    check_cuda(cudaDeviceSynchronize(), "init_rng_kernel_sync");

    init_spins_kernel<<<blocks_all, threads>>>(d_x, d_y, d_z, d_rng_states, n_sites);
    check_cuda(cudaGetLastError(), "init_spins_kernel_launch");
    check_cuda(cudaDeviceSynchronize(), "init_spins_kernel_sync");

    for (int sweep = 0; sweep < steps; ++sweep) {
        for (int parity = 0; parity < 2; ++parity) {
            mc_sweep_kernel<<<blocks_half, threads>>>(
                d_x, d_y, d_z,
                d_rng_states,
                size, parity,
                (float)J, (float)D, (float)B, (float)beta);
        }

        if (sweep % 1000 == 0 && saveframe == 1) {
            std::cout << "printing_frame..." << std::endl;
            frame = sweep / 1000;
            save_frame(d_x, d_y, d_z, n_sites, frame);
        }
    }

    std::vector<float> h_x(n_sites), h_y(n_sites), h_z(n_sites);

    check_cuda(cudaMemcpy(h_x.data(), d_x, n_sites * sizeof(float), cudaMemcpyDeviceToHost), "cudaMemcpy_D2H_x");
    check_cuda(cudaMemcpy(h_y.data(), d_y, n_sites * sizeof(float), cudaMemcpyDeviceToHost), "cudaMemcpy_D2H_y");
    check_cuda(cudaMemcpy(h_z.data(), d_z, n_sites * sizeof(float), cudaMemcpyDeviceToHost), "cudaMemcpy_D2H_z");

    double total_energy = 0.0;
    for (int i = 0; i < n_sites; ++i) {
        int row = i / size;
        int col = i % size;

        int n0 = ((row + 1 + size) % size) * size + col;
        int n1 = ((row - 1 + size) % size) * size + col;
        int n2 = row * size + ((col + 1 + size) % size);

```

```

    int n3 = row * size + ((col - 1 + size) % size);

    double xi = h_x[i], yi = h_y[i], zi = h_z[i];
    double e = 0.0;
    e -= J * (xi*h_x[n0] + yi*h_y[n0] + zi*h_z[n0]) - D * (zi*h_x[n0] - xi*h_z[n0]);
    e -= J * (xi*h_x[n1] + yi*h_y[n1] + zi*h_z[n1]) + D * (zi*h_x[n1] - xi*h_z[n1]);
    e -= J * (xi*h_x[n2] + yi*h_y[n2] + zi*h_z[n2]) - D * (yi*h_z[n2] - zi*h_y[n2]);
    e -= J * (xi*h_x[n3] + yi*h_y[n3] + zi*h_z[n3]) + D * (yi*h_z[n3] - zi*h_y[n3]);
    total_energy += 0.5 * e + B * zi;
}

auto end = std::chrono::steady_clock::now();
double n_ops = (double)size * size * steps;
double sec_per_step = (end - begin).count() / 1000000000.0 / n_ops;

// Print to stdout as before
std::cout << sec_per_step << "_sec/step" << std::endl;

// Append size + timing to CSV
append_timing(outfile, size, sec_per_step);

if (saveframe == 1) {
    save_spins(h_x, h_y, h_z, size, "frames/test_cuda.csv");
}

cudaFree(d_x);
cudaFree(d_y);
cudaFree(d_z);
cudaFree(d_rng_states);
return 0;
}

```